

Which dependencies are frequently missed by SBOM generation tools.

01

Background Exploration

1.1

Metadata Files that Can be Used by SBOM Tools

1.2

Which Metadata Files SBOM Tools Use

1.3

Defining the various types of dependencies and exploring which types SBOM tools typically fail to capture.

02

Zooming in on Transitive Dependencies

2.1

What does the Minimum Elements say about Transitive Dependencies?

2.2

What does the CycloneDX Specification say about Transitive Dependencies?

Insights from Syft sbom generation

How Syft Generates SBOMs:

1. Performs Static Analysis on input sources (container images, directories, files).
2. Uses ecosystem-specific "Catalogers" to parse known file types and structures.
3. Identifies components based on manifests, lock files, installed artifacts, binary formats, etc.
4. Crucially: Does NOT execute code, build tools, or invoke package managers (pip, npm, mvn, etc.).

Transitive Dependencies:

1. Lock Files: Excellent accuracy when lock files (package-lock.json, poetry.lock, yarn.lock, etc.) are present, as these contain the fully resolved dependency tree (including transitives).
2. Installed Artifacts: Effective when scanning directories containing installed packages (node_modules, Python site-packages) as these reflect the resolved state (often installed via a lock file).

Transitive Dependencies - Potential Gaps

1. Manifest-Only Analysis: If only basic manifests (requirements.txt, setup.py, library pom.xml) are found without lock files or installed packages, Syft typically lists only the directly declared dependencies. It cannot determine the transitive dependencies pulled in by those direct dependencies because it doesn't run the resolver.
2. Other Static Limitations: Difficulty fully resolving dependencies from standard JAR MANIFEST.MF files or reliably identifying statically linked libraries in binaries.

Potential Research Directions

1. Develop a framework that optionally invokes package managers (pip, npm, mvn) within a secure, isolated environment (e.g., container, sandbox) based on found manifests.
2. Develop a system where Syft (or a post-processor) queries public package repositories (PyPI, Maven Central, npm registry) using their APIs for the declared dependencies of the components found statically. Recursively query for the dependencies of those dependencies.
3. Research and advocate for standards or best practices where build systems embed richer, standardized dependency metadata (including hashes and resolved transitive dependencies) directly into build artifacts (e.g., JARs, binaries) in a way that tools like Syft can easily parse.

A deep dive into transcient dependencies

Existing methodologies

1

Using existing pip tools to populate requiemments.txt [1]

Pros:

- 1.Can integrate with existing SBOM generation tools
- 2.Uses existing dependency resolving mechanisms in pip

Cons:

- 1.Blackboxes the logic to pip tools
- 2.Does not display the hierarchal structure of transcient dependencies

2

Extracting transcient dependencies recursively by looking up the metadata

Pros:

- 1.Can integrate with pypi's current metadata fields
- 2.Requires only a few changes to existing SBOM tools

Cons:

- 1.Metadata is not available for some dependencies like non python depedencies

3

Extracting transcient dependencies recursively by looking up github repositories

Pros:

- 1.Can be modified to utilise any existing SBOMs as stated in the proposed PEP 765 standard

Cons:

- 1.Requires finding the source code dynamically
- 2.Requires multiple changes from current SBOM generation tools

